The Arch + dwl Media Workstation Book

Every concept, every package, every config file — with worked examples.

Companion to **arch-dwl-setup.sh** · 13 October 2026

The Arch + dwl Media Workstation Book

- The Arch + dwl Media Workstation Book
  - 1. What this is and why
    - 1.1 The stack in one paragraph
    - 1.2 Philosophy
    - 1.3 How to run the script
  - 2. The script: phases and environment variables
    - 2.1 Phases, in order
    - 2.2 Installing Arch for this script (archinstall, recommended settings)
    - 2.3 Environment variables
  - 3. Codec concepts: containers, codecs, and the modern set
    - 3.1 The one distinction that matters
    - 3.2 The video codec families you will meet
    - 3.3 Audio codecs you will meet
    - 3.4 Image codecs
    - 3.5 What Arch's ffmpeg already contains
    - 3.6 The packages and why each exists
    - 3.7 Worked examples
    - 3.8 How modern codecs actually compress (the 10-minute version)
    - 3.9 Containers compared
  - 4. Hardware decode and performance
    - 4.1 What hardware decode is
    - 4.2 Checking it works
    - 4.3 Why 120 Hz matters for films
    - 4.4 Gaming performance notes
    - 4.5 The decode pipeline, stage by stage
    - 4.6 When hardware decode fails, and why
  - 5. Colour and your monitor
    - 5.1 The concepts in five minutes
    - 5.2 Your exact monitor: LG 24G411A-B
    - 5.3 Your mpv.conf, line by line
    - 5.4 Colour discipline beyond mpv
    - 5.5 Technical definitions (colour)
    - 5.6 Going further with mpv (when you're ready)
  - 6. The desktop: dwl, bar, launcher, quick settings
    - 6.1 dwl in one page
    - 6.2 The bar (waybar, floating, 34 px)
    - 6.3 Launcher (rofi, Super+P)
    - 6.4 Quick settings (Super+X) and keybinds
    - 6.5 Wallpaper manager (dwl-wallpaper)
    - 6.6 The help menu (dwl-help, Super+/)
  - 7. Office, PDFs, ebooks, images
    - 7.1 The suite and its logic
    - 7.2 The PDF toolkit (replaces every "free PDF website")
    - 7.3 Worked examples
    - 7.4 Images: view, convert, compress
  - 8. Torrents without torrent websites
  - 9. Gaming on this machine
  - 10. Hardware: RGB, mouse, sound, cooler
    - 10.1 Sound fundamentals (the 5-minute version)

- 11. LazyVim, in depth
  - 11.1 What LazyVim is (and is not)
  - 11.2 How it works: the three layers
  - 11.3 The keymap language (memorize these ten)
  - 11.4 LSP, treesitter, formatting — the pipeline
  - 11.5 Extras and your AI setup
- 12. AI workflow: Claude Code + DeepSeek
  - 12.1 Claude Code (terminal agent)
  - 12.2 DeepSeek as a Claude backend (verified from DeepSeek's API docs)
  - 12.3 Avante in Neovim (DeepSeek or Claude)
  - 12.4 Key discipline
- 13. File-structure map: everything the script creates
- 14. The test suite: how it proves everything works
  - 14.1 --check-packages (before installing)
  - 14.2 --verify (after installing)
- 15. Troubleshooting FAQ
- 16. Arabic (read beautifully, write easily) + font variety
  - 16.1 The goal
  - 16.2 Why Arabic looks "weird" on a fresh install
  - 16.3 The Arabic fonts and what each is for
  - 16.4 Writing Arabic: the keyboard
  - 16.5 Arabic subtitles and ebooks
  - 16.6 Nerd font variety (terminal + bar icons)
- 17. Foundations: boot, services, storage, security
  - 17.1 Boot: firmware -> bootloader -> kernel -> session
  - 17.2 systemd units: system, user, and global
  - 17.3 Btrfs and snapshots, precisely
  - 17.4 Security: what each control actually does
- 18. Glossary and how to keep learning
  - 18.1 Glossary (terms used in this book)
  - 18.2 How to keep learning (a curriculum, in order)

A complete guide to your setup: every concept, every package, every config file, with worked examples.

This book documents the system built by `arch-dwl-setup.sh`: a minimal, fast Arch Linux workstation running the `dwl` Wayland compositor, tuned for watching movies and shows (full modern codecs, hardware decode, correct colour on your IPS monitor), light office work, gaming, and development with LazyVim + AI tooling. Everything is themed in Dracula and verified by an automated test suite.

---

1. What this is and why

1.1 The stack in one paragraph

Arch Linux gives you a rolling, minimal base with the newest packages. `dwl` is a Wayland port of `dwm`: a tiny, fast, tiling window manager you compile yourself, so it can be patched and themed exactly how you want. On top of it, this setup layers: hardware video decode (VA-API + Vulkan on your AMD GPU), a complete modern codec set, a minimal top bar (waybar), a launcher (rofi), notifications (mako), a terminal (foot), a network manager, a torrent search engine (Prowlarr), a full PDF toolkit, a light office suite, Steam gaming, and a LazyVim + Claude Code/DeepSeek development environment.

1.2 Philosophy

1. **Small and explicit.** Every package is named in one registry at the top of the script. Nothing is installed "by the way".
2. **Verified, not hoped.** `--check-packages` proves every package exists and its dependencies resolve before anything is installed. `--verify` runs real round-trip tests (encode/decode, split/merge, hardware decode) after installation.

3. **Idempotent.** Run the script ten times; the second run changes nothing. Configs are written only if absent (or with `THEME_FORCE=1` for re-theming).
4. **CLI-first, GUI where it earns its place.** The default tool is the terminal one; graphical apps (xournalpp, goverlay, nm-applet) exist only where a GUI is genuinely better.
5. **Your data stays yours.** Office files open locally, torrents come from a self-hosted search engine, PDF work needs no website.

1.3 How to run the script

```
chmod +x arch-dwl-setup.sh
./arch-dwl-setup.sh                      # full install
./arch-dwl-setup.sh --check-packages     # dry run: availability + dependency check
./arch-dwl-setup.sh --verify             # full test suite on the live system
SKIP=docker,verify ./arch-dwl-setup.sh   # skip phases
THEME=nord ./arch-dwl-setup.sh           # install with a different theme
THEME_FORCE=1 ./arch-dwl-setup.sh        # re-apply theme over existing configs
```

---

2. The script: phases and environment variables

2.1 Phases, in order

#	Phase	What it does
1	`pacman`	Enables colour, parallel downloads, `[multilib]`; full system upgrade; `base-devel`, timers
2	`paru`	AUR helper (`paru-bin`, prebuilt to skip a Rust compile)
3	`security`	`ufw` (deny incoming / allow outgoing), kernel hardening sysctls
4	`amd`	Mesa + RADV Vulkan + VA-API stack + `lact` daemon; `amdgpu.ppfeaturemask` kernel arg
5	`base`	PipeWire audio, portals, NetworkManager, clipboard, xwayland, session basics
6	`codecs`	ffmpeg + full modern codec set, mpv, yt-dlp, GStreamer plugins, disc libs, browser
7	`dwl`	Builds dwl from source at a pinned ref, applies patches, installs + rebuild hook
8	`visual`	waybar, rofi, foot, mako, swayosd, zsh + modern CLI tools
9	`qol`	Screenshots, recorder, pickers, file manager, images, cooler/RGB/sound tools
10	`fonts`	Noto, Nerd Font, Papirus icons, Arabic Naskh rules, zsh starter
11	`office`	AbiWord, Gnumeric, zathura + plugins, presenterm, xournalpp, PDF toolkit
12	`apps`	neovim, lazygit, node/npm, solaar, piper + AUR apps (LocalSend, Superseedr...)
13	`gaming`	Steam, gamescope, gamemode, mangohud, proton-ge, wine/protontricks
14	`torrent`	Prowlarr (search everything) + aria2
15	`docker`	Rootless Docker
16	`theme`	Dracula everything: configs, dwl colors/gaps/keybinds, wallpaper, greeter
17	`lazyvim`	Clone LazyVim starter + enable `ai.avante` / `ai.claudecode` extras
18	`ai`	`npm i -g @anthropic-ai/claude-code` (user-local prefix, no sudo)
19	`session`	`start-dwl` wrapper, autostart, wayland session file, minimal greeter
20	`services`	Enable greetd
21	`snapshots`	Timeshift on Btrfs + GRUB integration
22	`verify`	The full test suite (chapter 14)

2.2 Installing Arch for this script (archinstall, recommended settings)

Boot the Arch ISO, run `archinstall`, and choose:

Setting	Choose	Why
Profile	**Minimal**	No desktop environment — the script builds dwl + greetd itself. Picking GNOME/KDE means the script replaces its display manager later.
Mirrors	Your country (or Worldwide)	Faster downloads for the big first `-Syu`
Disk layout	Default automatic partitioning	—
Filesystem	**btrfs**	The Timeshift snapshot phase *requires* btrfs with the `@` subvolume layout, which is exactly archinstall's default
Encryption	LUKS if you want it	Works fine with everything (unlock at boot, snapshots still work)
Bootloader	**GRUB**	Snapshot entries appear in the boot menu via grub-btrfs (the script also supports systemd-boot, minus boot-menu snapshots)
Unified kernel images	Off (unless you know why you want them)	Kernel-parameter edits work differently under UKI
Swap	zram	No partition needed, modern default
Hostname	anything	—
Root password	set one	Recovery access
User	normal user, **ticked as sudoer**	The script *must* run as a non-root user with sudo — this is the single most important checkbox
Audio	**PipeWire**	The script enables PipeWire user units for everyone
Kernel	**linux**	RDNA4 needs >= 6.15, which every current option satisfies; `linux-zen` is an equal alternative for gaming
Network	**NetworkManager**	The script enables NM; choosing anything else invites a fight with systemd-networkd
Extra packages	leave empty	The script installs everything itself

Use wired ethernet during install if you can. After install: boot, log in as your user, connect network if needed, then run the one-liner from the repo README (or the local script). The script assumes nothing beyond a minimal install: it installs `curl` / `git` itself and its connectivity check works with zero extra packages.

2.3 Environment variables

Variable	Default	Meaning
`SKIP`	empty	Comma list of phases to skip
`AUR_REVIEW`	`0`	`1` = review every AUR PKGBUILD before building
`WITH_CHAOTIC`	`0`	Add the Chaotic-AUR binary repo (third-party trust)
`WITH_PPFEATUREMASK`	`1`	Unlock full amdgpu overclock controls (lact)
`WITH_XWAYLAND`	`1`	Build dwl with X11 app support
`DWL_REF`	latest tag	dwl git ref to build
`DWL_PATCHES`	`ipc genericgaps pertag swallow`	dwl-patches to apply
`KB_LAYOUT` / `KB_OPTIONS`	`us,ara` / `grp:alt_shift_toggle`	Keyboard layouts, Alt+Shift toggles
`TORRENT_PORT`	empty	ufw port to open for your torrent client
`WITH_BROWSER`	`0`	Opt-in Firefox install (VA-API decode + Widevine streaming)
`WITH_VLC`	`0`	Extra VLC install (mpv already covers everything)
`WITH_DISC_DECRYPT`	`0`	libdvdcss + libaacs (check your local law first)
`THEME`	`dracula`	`dracula` / `catppuccin` / `nord` / `tokyonight` / `none`
`THEME_FORCE`	`0`	`1` = overwrite theme-generated configs
`WITH_LAZYVIM` / `WITH_AI` / `WITH_TORRENT` / `WITH_GAMING`	`1`	Toggle those phases

---

3. Codec concepts: containers, codecs, and the modern set

3.1 The one distinction that matters

A video file is a **container** (a box) holding **streams** (the contents), each compressed by a **codec**:

- **Container** = `.mkv` (Matroska), `.mp4`, `.webm`, `.avi`. It says *how* streams are stored, synced and labelled (chapters, subtitles, multiple audio tracks).
- **Video codec** = H.264 (AVC), H.265 (HEVC), AV1, VP9, MPEG-2. It says *how* pictures are compressed.
- **Audio codec** = AAC, Opus, AC-3/E-AC-3 (Dolby), DTS, FLAC, MP3, Vorbis.
- **Subtitle tracks** = SRT, ASS/SSA (styled), PGS (Blu-ray bitmap), VobSub (DVD bitmap).

So "my movie won't play" is almost always one of: unknown video codec, unknown audio codec, missing subtitle renderer, or a broken container — not a broken player. The fix is codecs, not a different player.

3.2 The video codec families you will meet

Codec	Era	Where you meet it	Notes
H.264 / AVC	2003	Most Blu-rays, streams, old rips	Universal; every GPU decodes it in hardware
H.265 / HEVC	2013	4K Blu-rays, modern encodes	~50% smaller than H.264; 10-bit common
AV1	2018	YouTube, Netflix, new rips	Royalty-free; best quality per byte; your RX 9060 XT decodes it in hardware
VP9	2013	Older YouTube	Mostly legacy now; hardware decode everywhere
MPEG-2	1995	DVDs, old TV	Ancient; trivial to decode
Theora / Xvid	2000s	Very old files	Legacy; still supported

Rule of thumb: anything made after ~2020 is H.264, HEVC, or AV1. Your setup hardware-decodes all three — that is what "fast performance" means (chapter 4).

3.3 Audio codecs you will meet

AAC (streams, MP4), Opus (web, voice, music — the modern all-rounder), AC-3/E-AC-3 (DVD/Blu-ray surround), DTS/DTS-HD (Blu-ray), FLAC (lossless music), MP3/Vorbis (legacy music).

3.4 Image codecs

JPEG (universal), PNG (lossless), WebP (web), AVIF (AV1-based, tiny), JPEG XL / JXL (the future-proof archival format), JPEG 2000 (cinema/scans), HEIF (Apple cameras). Your setup reads and writes all of them (chapter 8).

3.5 What Arch's ffmpeg already contains

Arch builds ffmpeg with nearly every codec library enabled (verified from the official PKGBUILD): `libx264`, `libx265`, `libaom`, `dav1d`, `libsvtav1`, `librav1e`, `libvpx`, `libwebp`, `libjxl`, `libopenjpeg`, `libopus`, `libmp3lame`, `libvorbis`, `libspeex`, `libgsm`, `libtheora`, `libxvid`, plus VA-API/VDPAU/Vulkan hardware acceleration and `libass` (styled subtitles) + `libbluray` (Blu-ray structure). That is why the script does not need sketchy "codec packs": the distributor's ffmpeg **is** the full codec pack.

3.6 The packages and why each exists

Video libraries and tools

Package	Purpose
`ffmpeg`	The engine: decode, encode, remux, stream, screenshot — everything
`dav1d`	Fastest AV1 software decoder (used by everything, including ffmpeg)
`x264` / `x265`	The reference H.264/H.265 encoders + CLI tools
`svt-av1`	The fast production AV1 encoder
`libde265`	Standalone HEVC decoder (for non-ffmpeg apps)
`mpv`	Your player: libplacebo renderer on Vulkan, VA-API hardware decode
`yt-dlp`	Download from YouTube and a thousand other sites
`libplacebo` / `shaderc`	The GPU renderer + shader compiler mpv uses
`libass`	Styled ASS/SSA subtitles
`mediainfo`	"What is inside this file?" (codecs, bitrate, HDR, audio layout)
`mkvtoolnix-cli`	`mkvmerge` / `mkvextract` / `mkvinfo`: remux, add/remove/extract tracks

Image codecs

Package	Purpose
`libjxl` / `libavif` / `libheif` / `libwebp` / `openjpeg2`	JXL, AVIF, HEIF, WebP, JPEG 2000 support for ffmpeg, viewers, browsers

Audio

Package	Purpose
`opus` `flac` `lame` `libvorbis` `vorbis-tools` `wavpack`	Modern + lossless audio codecs and their CLI tools

GStreamer (for apps that are not mpv/ffmpeg)

Some apps (browsers, music players, PipeWire tools) decode through GStreamer, so the script installs the full plugin ladder: `base`, `good`, `bad`, `ugly`, plus `gst-libav` (ffmpeg's codecs inside GStreamer), `gst-plugin-va` (hardware decode), `gst-plugin-pipewire` (PipeWire bridge) and `gst-plugin-gtk`.

Optical discs

`libdvdread`, `libdvdnav`, `libbluray` (structure parsing). `libdvdcss` + `libaacs` (decryption) are official-repo packages now, but stay behind `WITH_DISC_DECRYPT=1` because breaking disc encryption is regulated in many countries — check your local law.

3.7 Worked examples

```
# What is in this file?
mediainfo Movie.2026.2160p.WEB-DL.mkv

# Remux MP4 -> MKV and add an external subtitle track
mkvmerge -o out.mkv Movie.mp4 subs.srt

# Extract the English subtitle track (track ids from mkvmerge -i)
mkvmerge -i out.mkv
mkvextract tracks out.mkv 2:english.srt

# Download the best quality and play it
yt-dlp -f 'bv*[height<=1080]+ba/b' 'https://…' -o '%(title)s.%(ext)s'

# Re-encode anything to efficient AV1 (software, slow but tiny)
ffmpeg -i in.mkv -c:v libsvtav1 -preset 6 -crf 32 -c:a copy out-av1.mkv

# Take a frame-accurate screenshot
ffmpeg -ss 00:42:10 -i Movie.mkv -frames:v 1 shot.png
```

3.8 How modern codecs actually compress (the 10-minute version)

Video compression rests on three ideas. Learn these and every codec makes sense:

1. **Chroma subsampling.** Human eyes resolve brightness far better than colour. So video stores full-resolution luma (Y, brightness) but quarter-resolution chroma (Cb/Cr, colour) — written `4:2:0`. A 1080p frame is ~3 MB raw RGB but ~1 MB in YUV 4:2:0 before any further compression. (This is also why red text on blue backgrounds always looks mushy.)
2. **Prediction + residual.** Instead of storing each frame, the encoder stores *keyframes* (**I-frames**, complete pictures) and, between them, *predicted* frames: **P-frames** (predicted from past frames: "this block moved 3 pixels left") and **B-frames** (predicted from past *and future* frames). Only the *difference* (residual) plus motion vectors are stored. A group of pictures (GOP) is typically 1 I-frame followed by 50–250 P/B frames — which is why seeking is instant to some points and slow to others, and why cutting video precisely requires re-encoding.
3. **Transform + quantization.** Residuals are transformed (DCT-like math) into frequencies, then *quantized*: small details are rounded away. The quantization strength is the quality knob. **CRF** (Constant Rate Factor: `--crf` in x264/x265, `-crf` in SVT-AV1) holds *perceived* quality constant and lets bitrate vary — this is what you want for archiving (CRF 18–23 for H.264, 24–32 for AV1). **CBR/2-pass** targets a *file size* instead — what streaming uses.

Profiles and levels are capability labels: `High@L4.1` (H.264) or `Main10` (10-bit HEVC) tell decoders what features to expect. Old hardware chokes on High 10 or 4K levels — one classic "won't play" cause that is really "can't decode in hardware, CPU too slow".

Why AV1 wins per byte: better partitioning (up to 128x128 blocks vs 64x64), better prediction modes, and no patent royalties (developed by the Alliance for Open Media: Google, Netflix, Mozilla, Amazon, Apple...). H.264/H.265 sit under patent pools (MPEG-LA and three

competing HEVC pools — the licensing mess is half of why AV1 exists). For *your* watching, the practical upshot from section 3.2 stands: your GPU decodes all three in hardware.

3.9 Containers compared

	MKV (Matroska)	MP4 (ISO BMFF)	WebM
Standard	Open (IETF RFC 9559)	ISO standard	Open, restricted profile of MKV
Video	anything	anything common	VP9/AV1 only
Audio	anything	anything common	Opus/Vorbis only
Subtitles	SRT/ASS/PGS/VobSub	mov_text (limited styling)	WebVTT only
Chapters/attachments/multiple editions	yes	weak	no
Best for	archiving, remuxing (use this)	phones/cameras/maximum compatibility	web embedding

Rule: **remux to MKV, never re-encode to change containers** (`mkvmerge` copies streams losslessly in seconds; re-encoding loses quality and takes hours).

---

4. Hardware decode and performance

4.1 What hardware decode is

Decoding 4K HEVC or AV1 in software burns CPU cores and battery. Your RX 9060 XT has fixed-function decoder blocks (VCN) that do it for milliwatts. The pipeline is: **mpv/ffmpeg -> VA-API (Linux video API) -> Mesa's RADV/RadeonSI driver -> GPU hardware**. Vulkan video (`--hwdec=vulkan`) is the newer equivalent path.

`hwdec=auto-safe` in your mpv.conf tries hardware decode and silently falls back to software if a clip needs something the hardware cannot do (e.g. 12-bit, exotic profiles). You never get a black screen; you get the fastest working path.

4.2 Checking it works

```
vainfo --display drm --device /dev/dri/renderD128   # lists H264/HEVC/VP9/AV1 profiles
vulkaninfo --summary | grep -i radv                 # Vulkan via RADV
mpv --hwdec=vaapi --vo=gpu-next Movie.mkv           # watch; press 'i' for stats (shows hwdec)
```

`--verify` does all of this automatically, including timing software vs hardware decode on generated H.264/HEVC-8/HEVC-10/VP9/AV1 clips.

4.3 Why 120 Hz matters for films

Films are 24 fps. A 60 Hz monitor cannot show each frame for an equal time (24 does not divide 60), so playback micro-stutters ("judder"). 120 Hz = 24 x 5 exactly: every frame lasts 5 refreshes. `video-sync=display-resample` in your mpv.conf locks pacing to the display. Set your LG to 120 Hz (OSD overclock or `wlr-randr --output DP-1 --mode 1920x1080@120Hz`) and films glide.

4.4 Gaming performance notes

- `gamemoderun %command%` as a Steam launch option: sets CPU governor to performance, I/O priority, and tells the compositor to unredirect.
- Gamescope (`gamescope -W 1920 -H 1080 -r 120 -- %command%`) gives games their own micro-compositor: upscaling/FSR, frame limiting, and clean alt-tab under dwl.
- MangoHud (`MANGOHUD=1 %command%` , configured in `goverlay`): FPS/frametime overlay to verify smoothness objectively.
- Proton: enable Steam Play for all titles, prefer `GE-Proton` (installed as `proton-ge-custom-bin`) — it carries newer media/codec fixes than stock Proton.

4.5 The decode pipeline, stage by stage

When you press play on an HEVC file, this happens — knowing the stages tells you exactly which log line to read when something breaks:

1. **Demuxing** (ffmpeg `libavformat`): the MKV is split into compressed H.265 + E-AC-3 + subtitle packets. Failures here = "Invalid data found" = broken container, fix with `mkvmerge`.
2. **Decoder selection** (`--hwdec=auto-safe`): mpv asks VA-API "can you do HEVC Main10?" If yes, compressed packets go straight to the GPU (stage 4). If no, software decode (stage 3).
3. **Software decode** (ffmpeg `libavcodec`, or `dav1d` for AV1): CPU reconstructs full frames. 4K AV1 in software wants ~6 fast cores; this is the stage that stutters on weak machines.
4. **Hardware decode** (VCN block via VA-API): fixed-function silicon outputs decoded frames into GPU memory — near-zero CPU, near-zero power.
5. **Rendering** (libplacebo, `vo=gpu-next`): scaling (lanczos/spline), dithering to your 8-bit panel, subtitle burn-in, OSD, tone-mapping if HDR. Then presentation synced to the 120 Hz clock (`video-sync=display-resample`).

`--hwdec=auto-safe` vs `auto` vs `vaapi`: `auto` tries hardware even for formats the driver claims dubiously (occasional green screens); `auto-safe` blacklists the risky paths; naming `vaapi` forces it and errors out instead of falling back. The script's default (`auto-safe`) is the right one — override per-file only to experiment (`mpv --hwdec=no file.mkv` forces software, useful to isolate a GPU bug).

4.6 When hardware decode fails, and why

Symptom	Likely stage	Fix
Green/pink corruption	4 (driver bug on an exotic profile)	`--hwdec=auto-safe` (already default) or update mesa
Stutter on one specific file	3 (fell back to software: 12-bit, AV1 film grain, 8K)	Check with `i` (stats overlay shows `hwdec: no`); pre-transcode that file
`Cannot load libva` / no profiles in `vainfo`	driver missing	Re-run the codecs/amd phases; you booted the wrong kernel
Everything stutters including H.264	5 (compositor/renderer)	Check MangoHud-style frame stats; `Ctrl+H` in mpv toggles hwdec live to compare

---

5. Colour and your monitor

5.1 The concepts in five minutes

- **Gamut** = the range of colours a panel can show. Yours covers ~99% of **sRGB** (the web's standard). Wide-gamut panels (P3, AdobeRGB) show richer reds/greens but need colour management to avoid looking neon.
- **Gamma / EOTF** = how brightness values map to light. sRGB has its own curve; most panels approximate gamma 2.2. For your panel, mpv's defaults are already right.
- **Bit depth** = steps per channel. 8-bit = 256 steps; banding appears in smooth gradients (skies). 10-bit video (HEVC Main10, AV1) carries finer steps; the GPU dithers them down to your 8-bit panel gracefully.
- **HDR vs SDR.** HDR needs 600+ nits and 10-bit. Your panel is 250 nits / 8-bit: it *accepts* an HDR10 signal and tone-maps it down, but it is an SDR panel. Real HDR tone-mapping in mpv only matters if you feed it HDR files — it maps them to SDR correctly by default.

5.2 Your exact monitor: LG 24G411A-B

23.8-inch IPS, 1920x1080, ~99% sRGB, 250 cd/m2, ~1000:1 contrast, 8-bit, 120 Hz native (144 Hz overclock), 1 ms MBR, FreeSync/G-Sync compatible, HDR10 *input*. Conclusion from research: this is a standard-gamut sRGB panel. Correct setup = (1) sRGB-ish picture mode in the OSD, (2) HDR **off** in the OSD, (3) 120 Hz, (4) mpv defaults (gpu-next + Vulkan). No ICC profile needed — profile loading exists in your mpv.conf (commented) for the day you buy a wide-gamut display.

5.3 Your mpv.conf, line by line

- `vo=gpu-next`, `gpu-api=vulkan` — the modern high-quality renderer on your AMD GPU (best scaling, dithering, tone-mapping).
- `hwdec=auto-safe` — hardware decode, software fallback (chapter 4).
- `video-sync=display-resample` — judder-free pacing at 120 Hz.
- `interpolation=no` — no soap-opera effect; films stay films.
- `ao=pipewire` — audio straight into PipeWire.

- `keep-open=yes`, `save-position-on-quit=yes` — resume where you left off.

5.4 Colour discipline beyond mpv

- The theme phase never touches colour *accuracy*: terminal/UI colours are cosmetic; video pixels go through mpv's managed pipeline.
- ArgyllCMS (`dispcal`, `dispread`, `dispwin`) is installed for the day you buy a colorimeter: measure the panel, install the resulting profile to `~/.local/share/icc/monitor.icc`, uncomment the two lines in mpv.conf.
- Night light: `gammastep -O 3500` (toggle from quick settings, Super+X) warms the screen after dark without touching video accuracy during the day.

5.5 Technical definitions (colour)

- **Colorspace** = primaries + white point + transfer function, bundled. sRGB, Rec.709 (HD video — same primaries as sRGB, slightly different transfer), DCI-P3 (cinema, wider), Rec.2020/BT.2020 (UHD container, widest). Your panel covers sRGB; a P3 panel shows ~25% more saturated reds/greens.
- **Primaries** = the exact red/green/blue the display mixes. **White point** = what "white" is (D65, daylight-ish, is the standard).
- **EOTF/OETF** = electro/optical transfer function: the math mapping signal values to emitted light. sRGB has a piecewise curve; gamma 2.2 is a pure power law most panels approximate.
- **SDR vs HDR, precisely.** SDR: ~100 nits reference, 8-bit, BT.709. HDR: PQ (ST.2084, *absolute* brightness up to 10,000 nits) or HLG (*relative*, broadcast-friendly), 10-bit+, BT.2020 container. **Diffuse white** (a white card, not a light source) sits at ~203 nits in HDR grading — everything above that is specular highlight, which is why tone-mapping matters.
- **Tone mapping** = compressing HDR brightness into an SDR panel's range. mpv's `--tone-mapping` operators (check `mpv --help` for your build): `clip` (hard cut, crushes highlights), `reinhard` / `hable` (filmic curves, pleasant rolloff), `mobius` (balanced default-ish), `bt.2390` (the broadcast standard), `spline` (modern, parametric). Defaults are correct for your SDR panel; touch these only with HDR files on an HDR display.
- **Dithering** = adding tiny noise when reducing bit depth (10-bit video -> 8-bit panel) so gradients stay smooth instead of banding. libplacebo does this in high precision on the GPU — one of the quiet reasons `vo=gpu-next` looks better.
- **ICC profile** = a measured description of *your specific panel* (primaries + tone response from a colorimeter). Generic ("sRGB.icc") profiles describe an ideal, not your screen. Only load a profile made by measuring your monitor.

5.6 Going further with mpv (when you're ready)

- Press `i` during playback: stats overlay (codec, hwdec state, frame timing, dropped frames). Press `?`: the full key list.
- mpv supports **profiles** in `mpv.conf`: the manual ships a `[reduce-judder]` example combining `video-sync=display-resample` with `interpolation=yes` (motion interpolation — the soap-opera effect; try it once on animation, then decide you hate it).
- Per-type configs: `[extension.mkv]`, `[protocol.https]` sections auto-apply (e.g. larger cache for network streams).
- `Ctrl+H` toggles hardware decoding live — the fastest A/B test when a file looks wrong.

---

6. The desktop: dwl, bar, launcher, quick settings

6.1 dwl in one page

dwl is ~8k lines of C: a Wayland compositor that tiles windows like dwm. You compile it because (a) you choose the version/patches, (b) its config *is* C (`config.h`), (c) a pacman hook rebuilds it automatically when wlroots updates. Your patches: `ipc` (bar talks to dwl), `genericgaps` (12 px gaps), `pertag` (per-tag layouts), `swallow` (terminal replaced by the GUI app it launches). Key file: `~/.local/src/dwl/config.h`. After editing: `dwl-rebuild`, log out/in.

Three deliberate deviations from stock dwl, applied by the theme phase: the modkey is **Super** (stock dwl uses Alt), the launcher is **rofi in apps-only mode** (stock uses `wmenu-run`), and the terminal lives on **Super+Return** (stock puts zoom there). Everything below assumes Super.

6.2 The bar (waybar, floating, 34 px)

Left: workspace tags (9, active = accent colour, urgent = red) + window title. Center: now-playing (`mpris`, any player via `playerctl`; dims when paused). Right: weather in Celsius (`wttrbar --nerd`, hourly), ethernet (IP, turns red when offline), Bluetooth (click opens `blueman`), volume (click opens `pavucontrol`, dims when muted), clock **with seconds** (click opens the `gsimplecal` calendar), tray. Config: `~/.config/waybar/config` + `style.css`.

Design rules (the "DNA", applied across bar, menus and notifications): - **Icons are Nerd Font glyphs, not emoji.** They're single-colour characters from the Material Design set in JetBrainsMono Nerd Font, so they take on the text colour of whatever surrounds them and switch automatically with `THEME=`. Emoji are full-colour bitmaps that ignore the theme, render differently from font to font, and sit off the text baseline. - **Floating panels.** The bar sits 8 px off the screen edges as three rounded panels (10 px radius, 1 px border in the surface colour, 92% opaque background) instead of a flat full-width strip. Notifications use the same radius and border and appear just below the bar. - **A 4 px spacing rhythm.** Every padding and margin is a multiple of 4: 12 px horizontal padding inside each module, 4 px between tags, 8 px gaps between panels and from the screen edges. - **One accent colour, with states shown by colour.** The accent marks only the focused/active thing. Red means a problem (offline, urgent), the dim grey marks something inactive (paused, muted, Bluetooth off), and everything else stays in the normal foreground colour. - **One shared theme file.** `~/.config/dwl/theme.sh` holds the glyphs and the `wmenu` colour/font arguments. The quick-settings, help and wallpaper menus all read it, so the three popups always look the same.

6.3 Launcher (rofi, Super+P)

`rofi -show drun`: applications only — Papirus icon + name, no terminal-run line, no history. Config: `~/.config/rofi/config.rasi` (`drun-display-format: "{name}"`, 26 px icons, 8 entries, 12 px padding, 12 px radius).

6.4 Quick settings (Super+X) and keybinds

`dwl-quick` is a themed `wmenu` menu with an icon on every row (items are matched by number, so changing a label or icon can't break its action): brightness +-5%, volume up/down/mute (with swayosd on-screen sliders), ethernet status + `nmtui`, Bluetooth on/off + `blueman`, night light toggle, region/full screenshots (grim + slurp -> clipboard + satty), wallpaper next/picker, lock, power menu (`wlogout`). Notifications (mako) follow the Dracula theme: accent border, red for urgent, dim for low-priority — every helper (`dwl-screenshot`, `dwl-wallpaper`, brightness/volume via swayosd) notifies through it. All keybinds are 61-key friendly (no Print, no media keys required; XF86 keys are *also* bound for full-size keyboards):

Keys	Action
Super+Return / Super+P / Super+X	terminal / launcher / quick settings
Super+/	searchable help menu: every shortcut + runnable action
Super+Up / Down, Super+Shift+Up	volume up / down / mute
Super+Left / Right	brightness down / up
Super+Shift+S / Super+Shift+F	region / fullscreen screenshot
Super+W / Super+Shift+W	wallpaper menu / next wallpaper
Super+Shift+Q	quit dwl

6.5 Wallpaper manager (`dwl-wallpaper`)

`~/Pictures/wallpapers` is your library: `--add` copies files in, `--remove` deletes via menu, `--list` picks, `--browse` views in `imv`, `--next` / `--random` cycle. `--fetch` pulls wallhaven.cc's **live top-30 toplist**, renders a labelled thumbnail grid (`montage` + `chafa` right in foot), you type a number, and it downloads full-res and sets it. Needs a free API key once (`wallhaven.cc -> Settings -> API -> dwl-wallpaper --set-apikey`, stored 600-permissions at `~/.config/wallhaven.key`). The generator wallpaper (`~/.config/dwl/wallpaper.png`) is a dithered dark gradient with Dracula glows — no banding on your 8-bit panel.

6.6 The help menu (`dwl-help`, Super+/)

A searchable popup listing every shortcut plus runnable actions (open apps, run updates, toggle night light, open Prowlarr...). Type to filter, Enter to run — the fastest way to learn the system and the fastest launcher for anything you forgot the key for.

---

7. Office, PDFs, ebooks, images

7.1 The suite and its logic

LibreOffice is ~1 GB and slow to start. Your suite is ~70 MB and instant: **AbiWord** (word processor, reads/writes `.docx`), **Gnumeric** (spreadsheet, `.xlsx`/CSV round-trips byte-identical in the test suite), **zathura** (PDF + EPUB + CBZ comics + DjVu + PostScript via

plugins), **presenterm** (Markdown -> terminal slides -> PDF), **xournalpp** (annotate + sign PDFs with pen or typed text). Fonts that keep Word documents looking right: Liberation (Arial/Times metrics), Carlito (Calibri), Caladea (Cambria), plus US English spellcheck.

7.2 The PDF toolkit (replaces every "free PDF website")

Job	Command
Split pages 1-5 out	`qpdf in.pdf --pages . 1-5 -- out.pdf`
Merge files	`qpdf a.pdf --pages . 1-2 b.pdf --pages . 1-3 -- out.pdf`
Extract all images	`pdfimages -png doc.pdf prefix`
Split every page to files	`pdfseparate doc.pdf page-%d.pdf`
Join pages	`pdfunite a.pdf b.pdf out.pdf`
Text out	`pdftotext doc.pdf - | less`
Repair/compress	`mutool clean -ggg in.pdf out.pdf`
PDF from images	`magick *.png out.pdf`
Rotate/encrypt	`qpdf in.pdf --rotate=+90:1-3 -- out.pdf` / `qpdf --encrypt userpw ownerpw 128 -- in.pdf out.pdf`
Annotate + sign	`xournalpp doc.pdf` (draw, highlight, insert signature image, export)

7.3 Worked examples

```
# .docx -> PDF without opening anything
abiword --to=pdf --to-name=report.pdf report.docx

# CSV -> formatted XLSX -> PDF (exact test from the suite)
ssconvert data.csv data.xlsx
ssconvert data.xlsx data.pdf

# Slides from Markdown, then to PDF for sharing
presenterm slides.md          # present in the terminal
presenterm export slides.md   # -> slides.pdf

# Read a book / comic
zathura novel.epub            # EPUB plugin
zathura volume-01.cbz         # comics plugin
```

7.4 Images: view, convert, compress

`imv` views anything (including AVIF/JXL). `chafa` previews images *inside foot* (used by the wallpaper fetcher). ImageMagick does everything else:

```
magick photo.png -resize 1920x1080 -quality 92 photo.jpg   # resize + convert type
magick *.png album.pdf                                      # images -> PDF
oxipng -o max -s icon.png                                   # lossless PNG shrink
jpegoptim --max=82 --strip-all *.jpg                        # lossy JPG shrink, strip metadata
```

---

8. Torrents without torrent websites

Prowlarr (`http://localhost:9696` , enabled as a system service) aggregates hundreds of public and private indexers behind one search box. First visit: Settings -> General -> set authentication, then Indexers -> Add -> enable public ones (1337x, TorrentGalaxy, Nyaa for anime...). Copy a magnet into your client (`superseedr` from the apps phase) or `aria2c '<magnet>'` . Open your client's listen port once:

`sudo ufw allow 6881` (or set `TORRENT_PORT=6881` before running the script). `mediainfo` the result, remux with `mkvmerge`, and you never touched a torrent site. Alternatives documented but not installed: `jackett-bin`, `bitmagnet-bin`.

---

9. Gaming on this machine

Steam (multilib) + the performance stack from chapter 4. Practical setup: Steam -> Settings -> Compatibility -> enable Steam Play, set default to `GE-Proton Latest` (installed by the script). Per-game launch options for demanding titles: `gamemoderun mangohud %command%`. For stubborn games: `gamescope -W 1920 -H 1080 -r 120 -f -- %command%`. `protontricks` + `winetricks` fix missing-codec cutscenes (`protontricks <appid> mf` installs Media Foundation). `goverlay` tunes the MangoHud overlay visually. Your saves, controller (G305's sibling gamepads work via kernel `hid`), and audio all flow through the standard stack — no extra drivers needed on AMD.

---

10. Hardware: RGB, mouse, sound, cooler

- **RGB (`openrgb`).** First run needs the udev rules (shipped) + a reboot so the kernel exposes the devices; then OpenRGB sees RAM/GPU/motherboard LEDs. Keep effects static or breathing — rainbow puke costs CPU wake-ups.
- **Logitech G305 (`solaar` + `piper`).** The G305 stores DPI and button mapping in onboard memory. `solaar` shows battery and link status for the Lightspeed receiver; `piper` (libratbag) edits DPI steps and button actions persistently. Set it once in either tool; the settings survive reboots and OS reinstalls.
- **Good sound (`easyeffects`).** PipeWire-native EQ/effects. Start with the built-in "Loudness" or "Bass Boost" preset, then add an Auto Gain + Limiter chain for late-night films so whispers stay audible and explosions stay neighbourly.

10.1 Sound fundamentals (the 5-minute version)

Digital audio is **samples per second** (sample rate) × **bits per sample** (bit depth) × channels. CD = 44,100 Hz × 16-bit × 2ch ≈ 1,411 kbps; studio = 48,000/96,000 Hz × 24-bit. Human hearing tops out ~20 kHz, and Nyquist math says you need 2× the highest frequency — hence 44.1/48 kHz. Higher rates don't sound "better", they just move filter artifacts further from audibility.

The Linux audio stack, bottom to top: hardware (ALSA kernel drivers, one per sound card) -> **PipeWire** (the graph: apps output to nodes, mixed, resampled if rates differ, routed to devices; replaces both PulseAudio and JACK) -> your ears. `pavucontrol` / `pamixer` control volumes per app; `pw-top` shows the live graph. **Resampling** happens whenever a 44.1 kHz song meets a 48 kHz output — PipeWire's SpeexDSP resampler is transparent; don't chase "bit-perfect" unless you know why.

EasyEffects chain order matters: input -> **Auto Gain** (levels the volume between quiet dialogue and loud scenes) -> **Equalizer** (fix your headphones' curve, cut rather than boost) -> **Limiter** (ceiling so nothing clips) -> output. Save one chain per output (headphones vs speakers) and switch with two clicks. - **Cooler LCD (`liquidctl`) — honest status.** `sudo liquidctl list` shows supported devices (Kraken, Commander, many AIOs/pumps). **Jungle Leopard coolers have no official Linux driver** (their software is Windows-only) — this was researched, not assumed. Options: (1) configure the display once in Windows — most of these coolers *remember* the last mode; (2) try community drivers (`pycooler-digital-display`, CoolerLCDLinux) by matching your USB ID from `lsusb`; (3) use `liquidctl` anyway for every other fan/pump curve in the loop. The verify suite checks `liquidctl list` and warns instead of failing.

---

11. LazyVim, in depth

11.1 What LazyVim is (and is not)

LazyVim is not a fork of Neovim — it is a *configuration framework*. Plain Neovim is a fast editor with no opinions; LazyVim adds a curated plugin set, sane defaults, and a keymap language, while leaving every file in `~/.config/nvim` yours to change. The script installs the official *starter* template, so your config is the standard one the whole community documents.

11.2 How it works: the three layers

1. `init.lua` — a 2-line bootstrap: sets the leader key (space) and loads `lazy.nvim`, the plugin manager.
2. `lua/config/` — *your* settings: `options.lua` (line numbers, tabs, clipboard), `keymaps.lua` (your shortcuts), `autocmds.lua` (actions on events), `lazy.lua` (which plugin groups load).
3. `lua/plugins/` — per-plugin overrides. Files here *extend* LazyVim's defaults; you never edit LazyVim's own code (it lives in the plugin cache, updated with `:Lazy update`).

`lazy.nvim` loads plugins *lazily*: the Git plugin only loads in a repo, the Markdown renderer only in `.md` files. That is why a 40-plugin setup still starts in ~50 ms.

11.3 The keymap language (memorize these ten)

Leader is `space`. Everything is mnemonic: `<leader>f` = find/file, `<leader>s` = search, `<leader>c` = code, `<leader>g` = git, `<leader>u` = ui toggles, `<leader>w` = window, `<leader>q` = quit.

Keys	Meaning
`<leader><space>` / `<leader>ff`	Find files (Telescope)
`<leader>fg` / `<leader>/`	Live grep across the project (needs `rg`, installed)
`<leader>e`	File tree sidebar
`gd` / `K` / `<leader>ca` / `<leader>cr`	Go to definition / hover docs / code actions / rename (LSP)
`<leader>cf`	Format file (conform + LSP)
`<leader>gg`	lazygit inside Neovim
`<leader>l`	Lazy plugin manager; `<leader>cm` = Mason (LSP servers)
`<leader>aa` / `ac` / `ae`	Avante: ask / chat / edit with AI (chapter 12)

11.4 LSP, treesitter, formatting — the pipeline

- **Treesitter** parses code into a syntax tree (precise highlighting, smarter motions). Parsers auto-install per filetype (`tree-sitter-cli` is in your packages for manual `:TSInstall`).
- **LSP servers** (via Mason, `:Mason`) give go-to-definition, errors, completions. Install per language: `:MasonInstall pyright`, `lua-language-server`, `typescript-language-server` ...
- **Formatting/linting** runs on save through `conform.nvim`. Language packs are LazyVim *extras*: `:LazyExtras` -> enable `lang.python`, `lang.markdown`, etc.

11.5 Extras and your AI setup

`:LazyExtras` lists plugin groups. The script pre-enabled two in `lazyvim.json`:

- `ai.avante` — an in-editor AI panel (`<leader>aa` ask about selection, `<leader>ac` chat, `<leader>ae` apply an edit, `<leader>ap` switch provider, `<leader>am` switch model). Providers are configured in `lua/plugins/ai.lua` (you create this file): set `provider = "claude"` with `ANTHROPIC_API_KEY`, or add a `deepseek` vendor block with `DEEPSEEK_API_KEY` (chapter 12).
- `ai.claudecode` — drives an external Claude Code session from inside Neovim (send buffer/selection as context).

First run: open `nvim`, wait for the plugin install to finish (`:Lazy` shows progress), restart, then `:checkhealth` to confirm treesitter/LSP/clipboard (`wl-copy` bridge) are green.

---

12. AI workflow: Claude Code + DeepSeek

12.1 Claude Code (terminal agent)

`claude` (installed user-local via npm, `~/.local/bin`) is an agentic coder: it reads your repo, runs commands, edits files, and shows diffs for approval. Core loop:

```
cd ~/project
claude                        # interactive; /help lists slash commands
claude "add input validation to the signup handler"   # one-shot task
```

Useful habits: `/init` to generate a `CLAUDE.md` memory file per project; `--allowedTools` for unattended runs; keep `git` clean so every AI change is reviewable. It bills to your Anthropic account via `ANTHROPIC_API_KEY`.

12.2 DeepSeek as a Claude backend (verified from DeepSeek's API docs)

DeepSeek exposes an Anthropic-compatible endpoint, so Claude Code (and any Anthropic SDK tool) can run on DeepSeek models with two variables — **no code changes**:

```
export ANTHROPIC_BASE_URL=https://api.deepseek.com/anthropic
export ANTHROPIC_API_KEY=<your-deepseek-key>   # from platform.deepseek.com
```

Model mapping (automatic): names starting `claude-opus` -> `deepseek-v4-pro` (billed at V4 Pro price); `claude-sonnet` / `claude-haiku` (or anything unsupported) -> `deepseek-flash`. When a tool requests a Claude model name, you transparently get the mapped DeepSeek model.

12.3 Avante in Neovim (DeepSeek or Claude)

Create `~/.config/nvim/lua/plugins/ai.lua`:

```
return {
  {
    "yetone/avante.nvim",
    opts = {
      provider = "deepseek", -- or "claude"
      vendors = {
        deepseek = {
          __inherited_from = "openai",
          api_key_name = "DEEPSEEK_API_KEY",
          endpoint = "https://api.deepseek.com/v1",
          model = "deepseek-chat",
        },
      },
    },
  },
}
```

Then `export DEEPSEEK_API_KEY=...` in your shell (never in a file that gets committed). `<leader>ap` switches provider mid-session, `<leader>am` switches model.

12.4 Key discipline

API keys live in environment variables (or a password manager / `pass`), never in `lazyvim.json`, never in git. The script enforces this by *not* writing any key anywhere — it only installs the tools and enables the extras.

---

13. File-structure map: everything the script creates

Path	What	Written by
`/usr/local/bin/start-dwl`	Session env (keyboard, Wayland vars) + launches dwl	session phase (root)
`/usr/local/bin/dwl`, `dwl-rebuild`	Compositor binary + rebuild helper	dwl/theme phases
`/etc/greetd/config.toml`	Minimal greeter (user+pass only)	session phase (root)
`/etc/tuigreet/config.toml`	Greeter theme, hidden clock/widgets	theme phase (root)
`/etc/sysctl.d/99-hardening.conf`	Kernel hardening	security phase (root)
`/etc/pacman.d/hooks/dwl-rebuild.hook`	Rebuild dwl on wlroots updates	dwl phase (root)
`/boot/grub/grub.cfg` (+ `GRUB_THEME`)	Dracula boot menu	snapshots/theme (root)
`~/.config/dwl/autostart.sh`	Session services (bar, daemons, wallpaper)	session phase
`~/.config/dwl/wallpaper.png`, `env`	Generated wallpaper, Docker env	theme/docker phases
`~/.config/waybar/config`, `style.css`	Top bar layout + Dracula style	theme phase
`~/.config/foot/foot.ini`	Terminal font/padding/colors	theme phase
`~/.config/mako/config`	Notifications	theme phase
`~/.config/rofi/config.rasi`	Icons-only app launcher	theme phase
`~/.config/starship.toml`, `.zshrc`	Prompt + shell	theme/fonts phases
`~/.config/fastfetch/config.jsonc`	System info + bat logo	theme phase
`~/.config/gtk-3.0`, `gtk-4.0`, `qt6ct`, `qt5ct`	Dracula for GTK/Qt apps	theme phase
`~/.config/mpv/mpv.conf`	Player: Vulkan, hwdec, 120 Hz pacing	codecs phase
`~/.config/nvim/` (+ `lazyvim.json`)	LazyVim + ai extras	lazyvim phase
`~/.config/fontconfig/conf.d/99-arabic.conf`	Arabic Naskh preference	fonts phase
`~/.config/xdg-desktop-portal/`	Screenshare + file-picker routing	qol phase
`~/.local/bin/dwl-wallpaper`, `dwl-quick`, `dwl-screenshot`	Wallpaper manager, quick settings, shots	theme phase
`~/Pictures/wallpapers/`	Your wallpaper library	created on first use
`~/.local/share/icc/`	Drop a measured monitor profile here	codecs phase (dir)
`~/.config/wallhaven.key` (600)	Your wallhaven API key	created by you via `--set-apikey`

---

14. The test suite: how it proves everything works

14.1 `--check-packages` (before installing)

Reads the same package registry the installer uses; for each entry it asks the Arch API (repo? AUR? provides?) and then resolves the full dependency closure with `pacman -Sp`. Output lines: green `repo`, yellow `AUR`, cyan `group -> chosen`, red `MISSING`. Non-zero exit on any problem. Run it any time — it installs nothing.

14.2 `--verify` (after installing)

Seventeen sections, each a real functional check, not a version-string grep:

- **packages**: every registry entry satisfies `pacman -T` (installed + deps complete).
- **binaries**: ~90 commands exist *and* `ldd` shows no missing libraries.

- **codecs**: ffmpeg encoder/decoder matrix (x264/x265/SVT-AV1/AOM/VP9/Opus/JXL/WebP/JPEG2000...), **real** software-vs-hardware timed decodes on generated clips, **real** WebP/JPEG/JXL/JPEG2000 encode-decode round trips, **real** `mkvmerge -> mkvextract` round trip, a real GStreamer `decodebin` pipeline, and an mpv hardware-decode confirmation.
- **office/pdf**: Gnumeric CSV-XLSX round-trip byte check, AbiWord PDF export, qpdf split-merge page-count check, `pdfimages` extraction, `mutool info`, zathura plugin presence.
- **images/torrent/gaming/theme/hardware/ai/monitor**: functional checks for each (magick resize pixel check, prowlarr unit, steam/gamescope/mangohud presence, theme files non-empty + scripts parse, liquidctl device scan, claude binary + lazyvim.json extras, wlr-randr refresh report).

Result line: `N passed, N failed, N warnings, N skipped`. WARN means "reboot first / needs a graphical session / optional"; FAIL means genuinely broken. Run it once right after install, and again after the first reboot.

---

15. Troubleshooting FAQ

dwl bounces back to the greeter. Read `~/.cache/dwl.log`. Usual suspects: wlroots/dwl version mismatch (re-run with `DWL_RESET=1`), or a syntax error you introduced in `config.h` (run `dwl-rebuild` and read the compiler error).

No hardware decode. `vainfo --display drm --device /dev/dri/renderD128` must list profiles. If empty: you booted the old kernel (reboot), or `mesa` / `libva` didn't install (re-run codecs phase).

Judder in films. Set the monitor to 120 Hz; keep `video-sync=display-resample`. Check MangoHud-style: `mpv --msg-level=vo=v` shows frame timing.

Washed-out colours. Turn HDR *off* in the monitor OSD (your panel is SDR). Keep the sRGB picture mode. Never set a random ICC profile "to try".

`.docx` **looks wrong.** AbiWord covers ~90% of real-world documents. For pixel-perfect legal/contract files, use any browser's online Office viewer or install LibreOffice temporarily — the fonts (Liberation/Carlito/Caladea) already prevent the worst substitutions.

Arabic shows disconnected/weird. See chapter 16: install check is `fc-match 'sans-serif:lang=ar'` — it must print an Arabic face, and `--verify` enforces this.

No wifi. `nmtui` or the tray icon; `systemctl status NetworkManager`. If `systemd-networkd` fights it, disable one of them.

Prowlarr empty. You must add indexers (Settings -> Indexers -> Add) — search needs sources, the script cannot add accounts for you.

Steam game is slow. Launch options `gamemoderun mangohud %command%`; check MangoHud FPS; try GE-Proton; for old titles `protontricks <appid> mf`.

Greeter looks unthemed. Theme comes from `/etc/tuigreet/config.toml`; a TUI greeter cannot show wallpaper images (VT limitation) — wallpaper appears the moment dwl starts. If you truly want a graphical login, the AUR `regreet` package is the alternative.

Something failed to install. Scroll to "Packages that did NOT install" in the summary, then run `./arch-dwl-setup.sh --check-packages` to see whether the name still exists upstream.

How the script survives version conflicts (v3.0+). Each class of mismatch has its own guard:

Situation	What the script does
Stale `db.lck` from a crashed pacman	Removes it only if no pacman/paru process is running; otherwise waits up to 5 min
Mirror timeout / partial download	Every pacman call retries 3x, re-syncing databases between tries
Outdated keyring ("invalid PGP signature")	Refreshes `archlinux-keyring` before the full upgrade
Partial-upgrade library mismatch	Only ever runs full `-Syu` upgrades, never `-Sy pkg`
paru broken by a new libalpm	Official-repo packages install through pacman directly, so they never touch paru. paru is health-checked and rebuilt: the `-bin` package is tried first, then a source build if the binary is still too old for the new libalpm
`paru` vs `paru-bin` conflict prompt	Each variant is removed individually with `-Rdd`, then the script confirms none are left before reinstalling
dwl release ahead of the repos' wlroots	Checks the newest 8 dwl tags and picks the newest one whose required wlroots version is actually in the repos
Broken AUR package	Recorded as FAILED and listed in the summary; the run carries on
Not enough disk / wrong clock / not in `wheel`	Preflight stops the run (disk) or fixes/warns (clock, wheel) before anything is installed

Every install fails with `paru: error while loading shared libraries: libalpm.so.X`**.** Your `paru` was built against an older pacman and a system upgrade pulled the library out from under it. Fix: rebuild it, then re-run the script (it resumes safely):

```
cd /tmp && rm -rf paru-bin && git clone https://aur.archlinux.org/paru-bin.git && cd paru-bin && makepkg -si --noconfir
paru --version   # must print cleanly, no libalpm error
```

If the prebuilt binary still mismatches (maintainer lags behind pacman), build from source — it always links correctly: same steps with `https://aur.archlinux.org/paru.git`. Current script versions self-heal this (health-check + auto-reinstall + source fallback), but if you're reading this, you're on the run that found the bug.

---

16. Arabic (read beautifully, write easily) + font variety

16.1 The goal

Not an Arabic *interface* — an English system where Arabic text renders correctly everywhere (browser, subtitles, PDFs, ebooks, terminal) and Arabic typing is one keystroke away.

16.2 Why Arabic looks "weird" on a fresh install

Arabic letters change shape by position (initial/medial/final/isolated) and join with ligatures (لا). If the system falls back to a font without proper Arabic tables, you get disconnected, ugly, or tofu boxes. The fix has three parts — all in this setup:

1. **Real Arabic fonts** (below).
2. **Shaping engines**: HarfBuzz (foot, browsers, mpv/libass subtitles), FriBiDi (bidirectional text), Pango (GTK apps) — all pulled in as dependencies automatically.
3. **Fontconfig rules** (`~/.config/fontconfig/conf.d/99-arabic.conf`): when an app asks for `sans-serif:lang=ar`, Noto Naskh Arabic wins; Chromium/Electron apps that forget the language hint still get it as a weak fallback.

16.3 The Arabic fonts and what each is for

Package	Source	Use it for
`noto-fonts-extra`	official	Noto Naskh Arabic (UI/docs default), Noto Sans Arabic, Nastaliq
`ttf-scheherazade-new`	official	Elegant Quranic-style Naskh — long reading
`ttf-amiri`	AUR	The most beautiful Naskh for books and PDFs
`vazirmatn-fonts`	AUR	Clean Arabic/Persian UI font
`zathura-epub-mupdf` + readers	AUR/official	Arabic EPUBs render with full shaping in zathura

Check any time: `fc-match 'sans-serif:lang=ar'` must print an Arabic face (the verify suite enforces this; if it prints DejaVu, re-run the fonts phase).

16.4 Writing Arabic: the keyboard

`start-dwl` exports `XKB_DEFAULT_LAYOUT=us,ara` with `grp:alt_shift_toggle`: **Alt+Shift switches English/Arabic instantly** in every app (terminal, browser, AbiWord, rofi). No input-method daemon needed (Arabic needs no IME, unlike CJK). In foot/zsh, Arabic echoes right-to-left-ish in display order — cosmetic only; the bytes are correct. In AbiWord, paragraphs follow the first strong character (start a line with Arabic for RTL); change direction explicitly from Format -> Paragraph -> direction when needed.

16.5 Arabic subtitles and ebooks

`mpv` shapes Arabic ASS/SRT through libass + HarfBuzz automatically — if a subtitle file shows disconnected letters, the file itself is broken (bad rip), not your system; remux a proper track with `mkvmerge`. Arabic EPUBs open in `zathura` with the EPUB plugin; Arabic PDFs in `zathura` or `xournalpp`.

16.6 Nerd font variety (terminal + bar icons)

One Nerd Font is never enough — different glyphs render better in different families. Installed:

Font	Personality	Good for
JetBrains Mono Nerd (default)	Neutral, crisp	foot default, code
FiraCode Nerd	Wide, friendly	long sessions
CaskaydiaCove Nerd (Cascadia)	Windows-familiar	converts from Windows Terminal
Hack Nerd	Compact	small screens
Roboto Mono / Ubuntu Mono Nerd	Distinctive	taste
Nerd Fonts Symbols (proportional)	Icons only	waybar/rofi glyphs
Font Awesome (OTF)	Icon set	waybar custom modules

Switch foot's font in `~/.config/foot/foot.ini` (`font=FiraCode Nerd Font:size=11.5`), then close and reopen foot to apply it. Waybar/rofi already reference `JetBrainsMono Nerd Font`; point them at any installed family the same way.

---

17. Foundations: boot, services, storage, security

Everything the script's "boring" phases do, explained so you can reason about the machine instead of memorizing commands.

17.1 Boot: firmware -> bootloader -> kernel -> session

1. **Firmware (UEFI)** reads the ESP partition and launches the bootloader.
2. **Bootloader. GRUB** (recommended, what archinstall sets up) understands filesystems, themes, and — via `grub-btrfs` — boots straight into Timeshift snapshots, which is why the script themes it and regenerates its config. **systemd-boot** is simpler and faster but can't list snapshots (the script supports it for kernel args, with a warning where GRUB-only features are skipped). The bootloader loads the **kernel + initramfs** (+ `amd-ucode` microcode first — CPU bug fixes that must load before anything else).

3. **Kernel parameters** (`GRUB_CMDLINE_LINUX_DEFAULT` , e.g. `amdgpu.ppfeaturemask=0xffffffff`) are the kernel's command-line flags. After editing, GRUB needs `grub-mkconfig -o /boot/grub/grub.cfg` (the script does this automatically and only when something changed).
4. **greetd** starts on VT1 and runs the greeter; login opens your session, `start-dwl` sets the environment, dwl starts the compositor.

17.2 systemd units: system, user, and global

- **System units** (`systemctl enable greetd` , `NetworkManager` , `prowlarr`): run as root for the whole machine, start at boot.
- **User units** (`systemctl --user enable dockerd-rootless`): run as you, start at login, die at logout. Rootless Docker *must* be a user unit — that's the entire security model.
- **Global user units** (`systemctl --global enable pipewire.socket ...`): symlinked into every user's session so audio "just works" for any account without per-user setup.
- **Timers** (`paccache.timer` , `fstrim.timer`): cron-style scheduled jobs — weekly cache cleaning and SSD trim.

Rule: if it touches hardware for everyone, system unit; if it's yours (audio, Docker, apps), user unit.

17.3 Btrfs and snapshots, precisely

Btrfs is a **copy-on-write** filesystem: overwriting a file writes new blocks and updates pointers instead of modifying in place. Consequences: (a) **subvolumes** (`@` , `@home`) are separate filesystem roots sharing one pool — *not* partitions, so space is never "stuck" in the wrong one; (b) a **snapshot** is just a frozen set of pointers — instant, nearly free; (c) Timeshift's Btrfs mode = scheduled snapshots + a GRUB menu to boot into yesterday's system when today breaks. The baseline snapshot the script takes ("after arch-dwl-setup") is your "known good" restore point. LUKS encryption sits *below* Btrfs (whole-disk), so snapshots work identically on encrypted installs.

17.4 Security: what each control actually does

- **ufw** `deny incoming / allow outgoing` : the firewall drops unsolicited inbound packets (nobody can reach your services) while everything you start works normally. Torrents need one explicit hole (`ufw allow <port>`) because peers connect *to* you.
- **Kernel hardening (** `99-hardening.conf` **)**: `kptr_restrict` / `dmesg_restrict` hide kernel addresses from unprivileged users (harder to exploit kernel bugs); `unprivileged_bpf_disabled` + `bpf_jit_harden` lock down the packet-filter VM; `protected_symlinks/hardlinks/fifos/regular` defeat classic `/tmp` race attacks; `tcp_syncookies` + `rp_filter` blunt spoofing and SYN floods; disabling redirects/source-routing kills whole classes of MITM tricks.
- **What the script deliberately does NOT do**: no `ip_forward=0` (Docker/VPNs need forwarding), no user-namespace lockdown (breaks Chromium/Electron sandboxes and rootless Docker). Security that breaks your software gets disabled by you — so it isn't security.
- **Rootless Docker**: the daemon runs as your UID inside user namespaces (`/etc/subuid` ranges map container-root to an unprivileged host UID). Never join the `docker` group — that's root-equivalent and voids the model.

---

18. Glossary and how to keep learning

18.1 Glossary (terms used in this book)

- **ALSA** — kernel-level audio drivers; PipeWire talks to ALSA, apps talk to PipeWire.
- **AUR** — Arch User Repository: community build scripts (`PKGBUILD` s) compiled on your machine via `paru` . Official repos (`core` / `extra` / `multilib`) are prebuilt by Arch devs.
- **Chroma subsampling (4:2:0)** — storing colour at quarter resolution; see 3.8.
- **Compositor** — on Wayland, the display server *and* window manager in one process (dwl). It alone talks to the GPU and input.
- **CRF** — Constant Rate Factor: constant-*quality* encode mode; see 3.8.
- **Demuxing/remuxing** — splitting a container into streams / re-boxing streams without re-encoding.
- **Dithering** — anti-banding noise when reducing bit depth; see 5.5.
- **DRM/KMS** — kernel GPU display stack (Direct Rendering Manager / Kernel Mode Setting); what `wlr-randr` and `vainfo --display drm` talk to.
- **EOTF** — transfer function mapping signal to light (sRGB curve, PQ, HLG); see 5.5.
- **Gamut** — range of displayable colours; see 5.1.
- **GOP** — group of pictures between keyframes; see 3.8.
- **Hardware decode** — fixed-function GPU video decoding via VA-API/Vulkan; see ch 4.
- **ICC profile** — measured description of one specific panel; see 5.5.
- **Idempotent** — safe to re-run; second run changes nothing (this script's core property).

- **KMS** — see DRM/KMS.
- **libplacebo** — mpv's GPU renderer library (scaling, dithering, tone-mapping).
- **LSP** — Language Server Protocol: editors get IDE features from per-language servers; see 11.4.
- **MPRIS** — standard media-player remote interface; why the bar shows any player's song.
- **PKGBUILD** — AUR build recipe; review with `AUR_REVIEW=1` before compiling strangers' code.
- **Portal** — Wayland's brokered API letting sandboxed apps request screensharing/file-pickers safely.
- **Profiles/levels (video)** — decoder capability labels like High@L4.1, Main10; see 3.8.
- **RADV** — community AMD Vulkan driver inside Mesa (what `vulkaninfo` should report).
- **Remux** — change container without touching streams (seconds, lossless).
- **Subvolume** — Btrfs filesystem root sharing one storage pool; see 17.3.
- **Torznab** — the search API Prowlarr speaks to indexers.
- **Transcode** — decode + re-encode (slow, lossy); distinct from remux.
- **VA-API** — Linux video acceleration API (Intel/AMD); see ch 4.
- **VRR/FreeSync** — variable refresh rate: display matches GPU frame timing, no tearing.
- **Wayland** — display protocol where the compositor does everything; X11 apps run via XWayland.
- **xdg-shell/layer-shell** — Wayland protocols for app windows (xdg) vs bars/lockscreens (layer).
- **XWayland** — X11 compatibility layer for old apps (built into dwl with `WITH_XWAYLAND=1`).

18.2 How to keep learning (a curriculum, in order)

1. **Man pages first:** `man mpv` (yes, all of it — search `/tone-mapping`), `man pacman.conf`, `man systemd.unit`, `man btrfs-subvolume`. Everything in this book bottoms out in a man page.
2. **Arch Wiki, specifically:** Wayland, PipeWire, VA-API, Btrfs, Timeshift, NetworkManager, Bluetooth, mpv, Wine/Proton, dwm (concepts transfer to dwl).
3. **Exercises that teach:** (a) `mpv --hwdec=no` vs default on the same 4K file while watching `btop` — *feel* hardware decode; (b) `qpdf --show-npages` before/after a merge you do yourself; (c) break dwl's `config.h` on purpose, read the compiler error, fix it, `dwl-rebuild`; (d) boot into yesterday's Timeshift snapshot from GRUB, then boot back — proving recovery works *before* you need it.
4. **Read code, not just docs:** `~/.local/src/dwl/config.h` (8k lines that run your whole desktop), then one helper script (`dwl-wallpaper --fetch` is a complete curl+jq+imagemagick pipeline to study).
5. **When stuck:** `./arch-dwl-setup.sh --verify` tells you *which layer* broke (chapter 14); the FAQ (chapter 15) maps symptoms to stages. Ask forums with the failing section's output pasted — "verify_codecs: hardware decode FAILED for hevc10" gets answers; "videos don't work" doesn't.